Solution: Pacific Atlantic Water Flow

Problem Statement:

Given a matrix `m x n` that represents the height of each unit cell in a Island, determine which cells can have water flow to both the Pacific and Atlantic oceans. The Pacific ocean touches the left and top edges of the continent, while the Atlantic ocean touches the right and bottom edges.

From each cell, water can only flow to adjacent cells (top, bottom, left, or right) if the adjacent cell's height is less than or equal to the current cell's height.

We need to return a list of grid coordinates where water can flow to both the Pacific and Atlantic oceans.

Example 1:

- **Input**: [[1,2,3],[4,5,6],[7,8,9]]
- **Expected Output**: [[0,2],[1,2],[2,0],[2,1],[2,2]]
- **Justification**: The cells that can flow to both the Pacific and Atlantic oceans are
  - `[0,2]`:
    - To Pacific Ocean: Directly from `[0,2]` since it's on the top border.
    - To Atlantic Ocean: `[0,2]` -> `[1,2]` -> `[2,2]`.
  - `[1,2]`:
    - To Pacific Ocean: `[1,2]` -> `[0,2]`.
    - To Atlantic Ocean: `[1,2]` -> `[2,2]`.
  - `[2,0]`:
    - To Pacific Ocean: Directly from `[2,0]` since it's on the left border.
    - To Atlantic Ocean: `[2,0]` -> `[2,1]`.
  - `[2,1]`:
    - To Pacific Ocean: `[2,1]` -> `[2,0]`.
    - To Atlantic Ocean: `[2,1]` -> `[2,2]`.
  - `[2,2]`:
    - To Pacific Ocean: `[2,2]` -> `[1,2]` -> `[0,2]`.
    - To Atlantic Ocean: Directly from `[2,2]` since it's on the bottom-right corner.

Example 2:

- **Input**: [[10,10,10],[10,1,10],[10,10,10]]
- **Expected Output**: [[0,0],[0,1],[0,2],[1,0],[1,2],[2,0],[2,1],[2,2]]
- **Justification**: The water can flow to both oceans from all cells except from the central cell [1,1].

Example 3:

- **Input**: [[5,4,3],[4,3,2],[3,2,1]]
- **Expected Output**: [[0,0],[0,1],[0,2],[1,0],[2,0]]
- **Justification**: All the leftmost cells can have water flowing to both oceans. Similarly, top cells also satisfy the criteria.

Solution

Overview: The problem is essentially asking for matrix cells from which water can flow to both the Pacific and Atlantic oceans. The matrix is viewed as an elevation map. By starting from the borders of the matrix, which are directly connected to the oceans, we can perform a Depth First Search (DFS) inwards to determine which cells can flow water to each ocean. We can then compare the results for both oceans to identify cells that can flow to both.

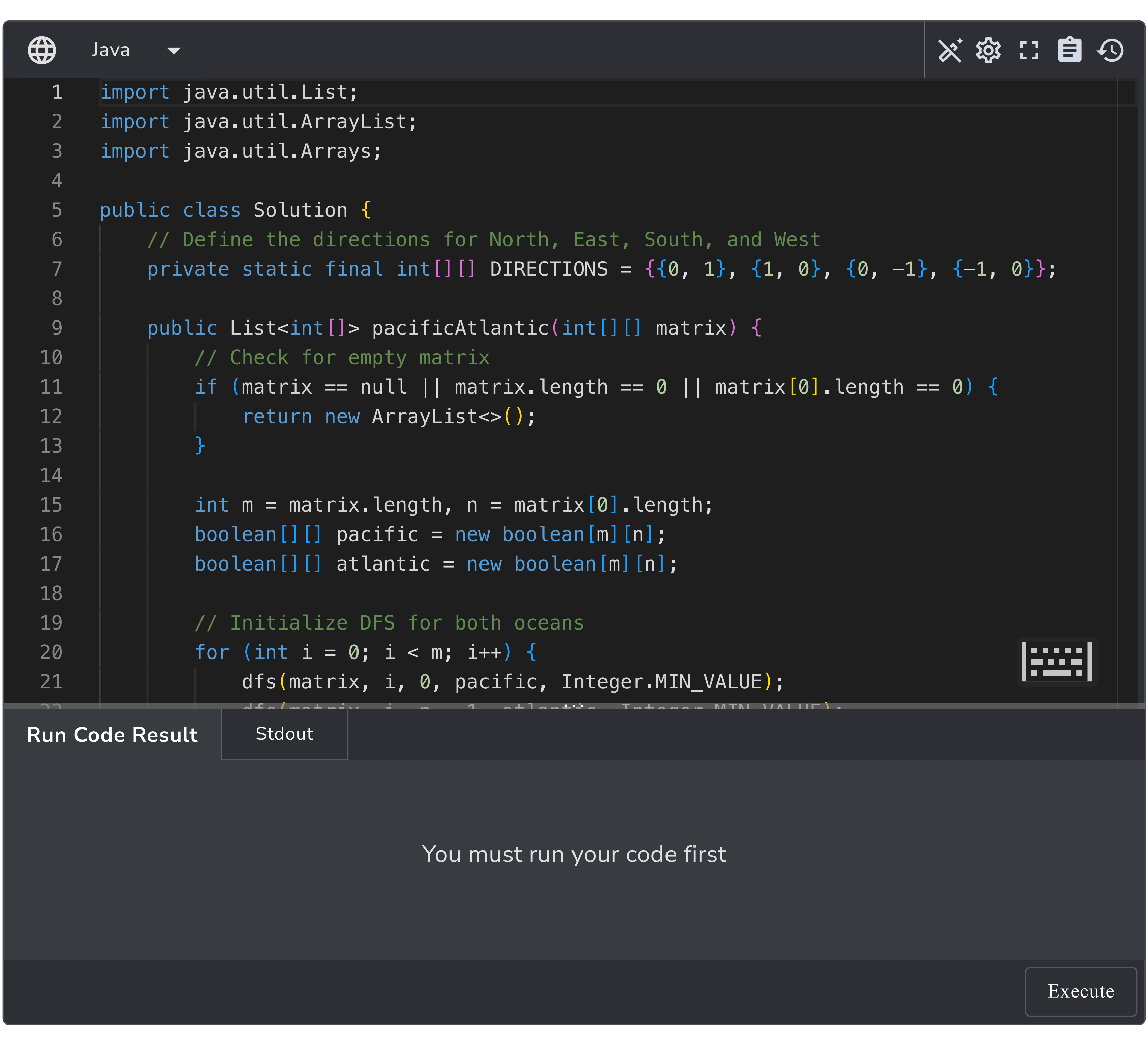
Detailed Explanation:

1. **Initialization:**
   - Begin by creating two matrices, `pacific` and `atlantic`, of the same size as the input matrix. These matrices will be used to mark the cells that can flow water to the respective oceans.
   - The edges of the matrix adjacent to the top and left borders are considered part of the Pacific, while those adjacent to the bottom and right borders are considered part of the Atlantic.
2. **DFS Traversal:**
   - Perform a Depth First Search (DFS) starting from each cell on the borders.
   - For the Pacific ocean, initiate DFS from the top and left borders. For the Atlantic ocean, initiate DFS from the bottom and right borders.
   - While traversing using DFS, move from a cell to its neighboring cells only if the neighboring cell's height is greater than or equal to the current cell. This ensures that water can flow in that direction (from high or equal elevation to higher elevations).
   - Mark cells as visited for each ocean as you traverse to prevent reprocessing.
3. **Result Compilation:**
   - After completing the DFS traversal for both oceans, iterate over the matrix to identify cells that were visited in both the `pacific` and `atlantic` matrices. These are the cells from which water can flow to both oceans.
   - Collect these cells and return them as the result.

Algorithm Walkthroug

1. **Initialization:**
   - We have our input matrix: `[[1,2,3],[4,5,6],[7,8,9]]`
   - Create two matrices `pacific` and `atlantic` with dimensions matching the input matrix, filled with `False` values. These will keep track of cells water can reach from each ocean.
   - Define the matrix boundaries: top and left for Pacific, and bottom and right for Atlantic.
2. **Starting from Borders:**
   - For the **Pacific** ocean:
     - Start DFS from the top border: `[0,0]`, `[0,1]`, and `[0,2]`.
       - For `[0,0]` and `[0,1]`, water cannot flow left or upwards as there's no cell in that direction. Only downwards or to the right is possible. But since the elevation increases in both these directions, water cannot flow.
       - For `[0,2]` (value 3), water can flow downwards to `[1,2]` (value 6) as 3 < 6.
     - Start DFS from the left border: `[0,0]`, `[1,0]`, and `[2,0]`.
       - Only `[2,0]` (value 7) can flow to `[2,1]` (value 8) as 7 < 8.
   - For the **Atlantic** ocean:
     - Start DFS from the bottom border: `[2,0]`, `[2,1]`, and `[2,2]`.
       - For `[2,0]` and `[2,1]`, water cannot flow downwards as there's no cell in that direction. Only upwards or to the left/right is possible. However, only `[2,1]` can flow to `[2,2]` as 8 < 9.
       - For `[2,2]` (value 9), water can flow upwards to `[1,2]` (value 6) as 9 > 6.
     - Start DFS from the right border: `[0,2]`, `[1,2]`, and `[2,2]`.
       - For `[0,2]`, water can flow downwards as already discussed.
       - For `[1,2]`, water can flow upwards to `[0,2]` and downwards to `[2,2]`.
3. **Identifying Common Cells:**
   - Iterate through the matrix and find cells where both `pacific` and `atlantic` matrices are `True`.
   - For our example, the cells are: `[0,2]`, `[1,2]`, `[2,0]`, `[2,1]`, and `[2,2]`.

Code



```java
import java.util.List;
import java.util.ArrayList;
import java.util.Arrays;

public class Solution {
    // Define the directions for North, East, South, and West
    private static final int[][] DIRECTIONS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

    public List<int[]> pacificAtlantic(int[][] matrix) {
        // Check for empty matrix
        if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
            return new ArrayList<>();
        }

        int m = matrix.length, n = matrix[0].length;
        boolean[][] pacific = new boolean[m][n];
        boolean[][] atlantic = new boolean[m][n];

        // Initialize DFS for both oceans
        for (int i = 0; i < m; i++) {
            dfs(matrix, i, 0, pacific, Integer.MIN_VALUE);
```

You must run your code first

Complexity Analysis

Time Complexity: O(m x n) where m is the number of rows and n is the number of columns in the matrix. This is because each cell is visited once for both oceans.

Space Complexity: O(m x n) due to the two additional matrices (for Pacific and Atlantic) to keep track of visited cells.

← Previous
Pacific Atlantic Water Flow (Medium)

Next →
Validate Binary Search Tree (Medium)

☐ Mark as Completed